

Unit-03

Delegates and String

3.1. Delegates

A delegate is a type in C# that allows you to reference methods with a particular parameter list and return type. Delegates are used to invoke methods indirectly and enable events and callback mechanisms. It acts as an intermediary, allowing methods to be passed as parameters and invoked at a later time. Delegates provide a way to implement callback methods and event handling in C# applications.

- **Single-cast Delegate:** Refers to one method.
- **Multicast Delegate:** Refers to multiple methods that execute sequentially.
- **Used in Event Handling:** Delegates define the event handler methods.

Syntax: public delegate returnType DelegateName(parameters);

Example:

```
public delegate void PrintMessage(string message);

public void ShowMessage(string message)
{
    Console.WriteLine(message);
}

PrintMessage print = new PrintMessage(ShowMessage);
print("Hello World");
```

3.2. Lambda Expressions and Its Implementation

A lambda expression in C# is a shorthand way to write anonymous methods. It allows you to define a method inline without needing a separate method definition. Lambda expressions are often used for short, single-use methods that can be passed as arguments, especially in LINQ queries and event handling.

Syntax: (parameters) => expression

Example:

```
Func<int, int, int> add = (x, y) => x + y;

Console.WriteLine(add(5, 6));
```

3.3. Event Handling

An event in C# is a mechanism to signal when an action or specific condition occurs. It is typically used for user interactions like button clicks, key presses, or system notifications. Events provide a way to decouple the publisher (the source of the event) from the subscriber (the listener or handler for the event), which enhances the flexibility and maintainability of the application.

Syntax: `public event DelegateName EventName;`

Example:

```
public delegate void Notify();

public event Notify OnNotify;

public void TriggerEvent()
{
    OnNotify?.Invoke();
}

public static void Main()
{
    Program program = new Program();

    program.OnNotify += () => Console.WriteLine("Event Triggered!");

    program.TriggerEvent();
}
```

3.4. String Manipulation and String Builder

String Manipulation:

C# provides powerful methods to manipulate strings, which makes it easy to work with text data. These methods allow you to perform operations such as concatenation, substring extraction, replacement, comparison, and more. Strings in C# are immutable, meaning that once a string is created, it cannot be modified. Any operation that appears to modify a string actually creates a new string.

Common Methods:

- **Concat:** Concatenates two or more strings.
- **Substring:** Extracts a part of a string.

- **Replace:** Replaces characters or substrings.
- **IndexOf:** Finds the index of a substring.

Example:

```
string str = "Hello, World!";
string result = str.Substring(7, 5);
Console.WriteLine(result);
```

StringBuilder:

StringBuilder is a class in C# that provides an efficient way to work with strings, especially when the string is modified repeatedly. Unlike regular strings, which are immutable, StringBuilder allows you to modify the content of a string without creating new instances, thus improving performance for operations like appending, removing, or replacing text multiple times.

Example:

```
StringBuilder sb = new StringBuilder("Hello");
sb.Append(", World!");
Console.WriteLine(sb.ToString());
```

3.5. Collections: Generic and non-generic

Collections in C# are used to store, manage, and manipulate groups of objects. They are an essential part of the C# programming language, allowing developers to work with data in a more structured and efficient manner. Collections are categorized into two main types:

Generic Collections and Non-Generic Collections.

Generic Collections:

Generic collections were introduced in C# 2.0 and provide type safety. They allow you to specify the type of objects stored in the collection, ensuring that only objects of the specified type can be added. Generic collections are part of the System.Collections.Generic namespace.

- **List<T>:** Represents a collection of objects that can be indexed.
- **Dictionary<TKey, TValue>:** Represents key-value pairs.
- **Queue<T>:** FIFO (First In, First Out) collection.
- **Stack<T>:** LIFO (Last In, First Out) collection.

Non-generic Collections:

Non-generic collections allow storage of objects of any type. However, since they are not type-safe, they require casting when retrieving elements, which may lead to runtime errors. They are part of the System.Collections namespace.

- **ArrayList:** A non-generic collection that can store any type of objects.
- **Hashtable:** A non-generic collection of key-value pairs.

Example:

Generic Collection (List<T>):

```
List<int> numbers = new List<int> {1, 2, 3, 4};  
numbers.Add(5);  
Console.WriteLine(numbers[0]);
```

Non-generic Collection (ArrayList):

```
ArrayList list = new ArrayList();  
list.Add(1);  
list.Add("Hello");  
Console.WriteLine(list[1]);
```